EXAM MAP

COMP9001 final: trace code, explain errors, design functions/classes, file I/O, exceptions, recursion; always answer in English comments.

Answer recipe: read spec → identify inputs/outputs/types → list edge cases → design alg → trace 1 sample → write clean Python.
Trace table: line/iteration, condition, variable changes, output. Include skipped branches and final return.
Common traps: '=' vs '=='; off-by-one; mutation vs rebinding; missing return; string/list immutability; integer div; scope; file close; unhandled empty input.

Task	Best pattern
parse args	`sys.argv[1:]` + validate length
repeated input	sentinel `while word!="quit"`
counting	dict `d[x]=d.get(x,0)+1`
transform	new list; avoid mutating unless asked
robust file	`try/except FileNotFoundError/ValueError`

W1 PROGRAM BASICS

Program: seq of statements executed top→bottom except control flow; **expression** produces value; **statement** performs action.

`print(x)` displays; `input()` returns **str**; comments start #; indentation defines blocks.

Style: meaningful names, snake_case, constants uppercase, 4 spaces, no magic numbers, short funcs.
Spec to code: nouns→data vars; verbs→functions; constraints→ifs; repeated action→loop; records→class/dict.
First bug hunt: missing colon, bad indent, quote mismatch, unclosed bracket, wrong filename, unused returned value.

Python evaluates RHS before assignment: `x=x+1` needs old x defined.

Operator prec: () then **, unary, * / // %, + -, comparisons, `not`, `and`, `or`.
Mini: `int(5/2)=2`; `5//2=2`; `5%2=1`; `bool("")=False`.

W2 TYPES & VARIABLES

Type: set of values + ops. Core: **int,float,bool,str,list,dict,tuple,NoneType**.

`type(x)`; casts `int()`, `float()`, `str()`, `bool()`; invalid cast raises `ValueError`.

Mutable: list/dict/set; **immutable**: int/float/bool/str/tuple. Mutation changes object; rebinding changes name.

Op	Meaning
`a=b`	both names point same object
`lst.append(x)`	mutates list in place
`lst=lst+[x]`	new list, name rebound
`s.upper()`	new string returned

Aliasing: if `b=a` and `a` list, `b.append(1)` visible through `a`. Use `a.copy()` for shallow copy.
String indexing: zero-based; negative index from end; slice `s[i:j]` excludes j.

Never compare floats for exact equality after arithmetic; use tolerance.

W3 CONDITIONALS

if/elif/else: first true branch runs; later branches skipped. Condition must be bool or truthy/falsy value.

`and` stops on first False; `or` stops on first True; `not` flips truth.

Boolean design: validate preconditions first; handle special cases before general cases; avoid duplicate tests.
Fencepost: <= includes endpoint, < excludes; tax/grade bands need ordered thresholds.

Pattern	Use
guard	`if bad: return None`
chain	mutually exclusive bands
nested	dependent decisions only
flag	remember if event found

Mini tax: if income ≤ 18200 tax 0; elif ≤ 45000 tax $(i-18200).16$; elif ≤ 135000 tax $4288+(i-45000).30$.
Flowchart: diamond=condition; rectangle=action; arrows show next statement; include loop-back edge.

W4 ITERATION

Loop: repeats block while condition holds or over sequence.

`for i in range(n)` gives $0..n-1$; `range(a,b,s)` excludes b; s may be negative.

while: initialise before loop; condition; body; update progress; postcondition. Missing update → infinite loop.
Loop cliches: count, sum, max/min, search, filter, transform, validate until sentinel.

Cliche	Invariant
sum	total = sum of processed items
max	best = largest seen so far
search	found iff target seen
filter	out contains kept processed items

Do not change loop variable to skip range; use `continue/break` only when it simplifies.
Trace: for `i=0..3`, even adds 1, odd adds 2; start 1 → 1+1+2+1+2=7.
Index loop when need position; direct loop when need value only.

W5 FUNCTIONS

Function: named reusable computation with params, local scope, optional return.
If no `return`, Python returns `None`; printing inside function is not returning.

Parameter passing: object reference copied. Mutating a passed list affects caller; rebinding param does not.
Design: signature → docstring/spec → base cases/errors → compute → return single clear result.
Scope: local vars die after call; global read ok, modify needs `global` but avoid for exam unless asked.

Kind	Example
default	`def f(x,n=0)`
variable positional	`*args`
keyword	`name=value`
higher-order	function arg/return

Mini: `def f(lst): lst.append(4); lst=[10]` leaves caller list with appended 4 only.

Pure functions easier to test: same input ⇒ same output, no hidden I/O/global mutation.

W6 COLLECTIONS

List: ordered mutable seq; **tuple**: ordered immutable; **dict**: key→value; **set**: unique unordered.

List ops: `append`, `extend`, `pop`, `remove`, `sort`; dict ops: `keys`, `values`, `items`, `get`.

`remove(x)` deletes first x and mutates; use copy or comprehension if original needed.
Count dict: `counts[w]=counts.get(w,0)+1`; then iterate sorted keys/items if order required.

Task	Code idea
second largest	track top2 distinct values
frequency	dict or `Counter`
dedupe	seen set, output list
nested data	list of dicts/classes

Comprehension: `[expr for x in xs if cond]`; concise but avoid if readability suffers.
Membership in list $O(n)$; dict/set average $O(1)$ for keys.

W7 CLASSES & OBJECTS

Class: blueprint; **object**: instance with attributes + methods. `self` is current object.

`__init__` constructs state; `__str__` printable text; `__len__`, `__eq__` support built-ins/operators.

Instance attr: `self.name`; **class var**: shared by all instances, access as `Class.var`.
Method call: `obj.m(x)` becomes `Class.m(obj,x)`; missing self is common error.
Encapsulation: methods enforce invariants, e.g. no negative price/gold/books.

OOP idea	Exam cue
constructor	attributes/defaults
inheritance	specialised product/fish/student
override	subclass display
composition	school has list of students

When asked for `len(obj)`, implement `def __len__(self): return len(self.items)`.

W8 FILE I/O

File: stream of text bytes decoded into lines. Always know mode: `'r'`, `'w'`, `'a'`.

`with open(path) as f:` auto-closes even if error; `for line in f` is memory-safe.

Parse line: strip newline → skip empty/validate → split → cast → accumulate.
`readlines()` not `read_lines()`; `int("3.0")` error; `print("Avg:"+str(avg))`.
Robust outputs: create output file only after all computations succeed; collect strings then write.

Exception	Likely cause
`FileNotFoundError`	bad path read
`ValueError`	bad cast
`ZeroDivisionError`	empty valid data
`IndexError`	missing argv/field

Line numbers usually start at 1 in specs; use `enumerate(f,1)`.

W9 FLOW CONTROL+

break: leave nearest loop; **continue**: jump to next iteration; **return**: leave function.
Use early `return` for guard clauses; use `break` after target found if no later data needed.
Sentinel loop: initialise input before/inside; stop exactly when sentinel seen; do not process sentinel.
Input validation: read → try cast → check range → repeat or report.
Nested loops: inner completes for each outer item; compare all pairs with `for i... for j... if i!=j`.

Pattern	Sketch
all pairs	nested loops skip self
manual substring	outer start, inner chars
command args	loop over `sys.argv[1:]`
anagram	sort chars or count dicts

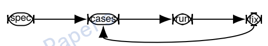
For manual search restrictions, avoid `in`, slicing, `find`; compare characters by index.

W10 TESTING & DEBUG

Testing: show program meets spec; **debugging**: locate/fix reason it does not.
Test set: normal, boundary, empty, singleton, duplicates, invalid type/text, missing file/arg, large-ish.

One test should assert expected output/return; choose edge cases from every branch.

Debug: reproduce → reduce input → inspect traceback line → form hypothesis → change one thing → rerun.



Traceback: bottom frame usually where exception raised; error type tells class of bug.

Bug	Check
NameError	scope/spelling/order
TypeError	wrong op/arity
SyntaxError	colon/bracket/indent
Logic	trace + invariant

W11 RECURSION

Recursion: function solves problem by calling itself on smaller input.

Must have base case + progress toward base + combine step. Missing base/progress → recursion error.

Base cases: empty list/string, length 1, n==0, target found/not found.
Recursive recipe: if base return simple answer; else compute first/last + recurse on rest; return combined answer.

Problem	Recursive form
factorial	$n!=n(n-1)!$, $0!=1$
palindrome	ends equal and middle palindrome
tree/network	search each friend/subnode
list sum	first + sum(rest)

State in recursion: avoid shared mutable default args; pass visited/path only if cycles possible.

Exam proof: identify smaller subproblem, base, and why termination occurs.

Mini: `find(name)` returns 0 if direct friend, else 1+child distance if found, else -1.

W12 ITERATORS & 2D

Iterator: object giving next values lazily; `for` calls iteration protocol.
2D list: list of rows; access `grid[r][c]`; dimensions `rows=len(grid)`, `cols=len(grid[0])`.

Build independent rows: `[[0]*cols for _ in range(rows)]`; avoid `[[0]*cols]*rows`.

2D traversal: for r in range(rows): for c in range(cols): action(grid[r][c]).

Task	Pattern
row sum	loop cols for fixed r
col sum	loop rows for fixed c
neighbours	bounds check r±1,c±1
transpose	out[c][r]=grid[r][c]

Ragged arrays: not every row same length; use `len(grid[r])`.
zip/enumerate: `enumerate(xs)` gives index,value; `zip(a,b)` pairs until shorter ends.

PAST PAPER CUES

21–25 finals: long tracing, flowcharts, file parsing, class methods, recursion, string/list processing, error identification.

When asked to identify errors: line no + error + fix + why. A corrected line alone is not full credit.

Program writing rubric: correctness against tests + style/readability; include helper funcs when spec has repeated subtasks.
Do not overuse libraries: if prompt restricts modules/keywords/slicing, obey exactly; violating restriction can zero the subquestion.

Prompt word	Expected evidence
trace	table and final output
robust	exceptions + edge cases
design class	attrs, init, methods, demo
recursive	self-call on smaller input
parse file	line-by-line validation

MICRO EXAMPLES

`second_largest`: use distinct values: top1=None, top2=None; update only if x differs from top1.

`manual_contains`: for start in range(len(s)-len(sub)+1), compare chars until mismatch.

`file average`: valid=[]; for line: try append(float(line)); except ValueError: skip; if not valid: report.

Final check: run mentally on empty/singleton/duplicate/invalid; match exact print format; no stray debug output.

CODE CLICHES

Min/max: init with first valid item, then update; if empty return/print error before using max.

`best = None`; for x in xs: if best is None or x < best: best=x

Search: found flag or early return; return index/value only after match.

`for i in range(len(xs)): if xs[i]==target: return i; return -1`

Filter: create out=[]; append only if predicate; never remove from list while iterating over same list.
Map/transform: `out.append(transform(x))`; preserve input unless mutation explicitly required.
Validate argv: if `len(sys.argv)<2`: print error; else loop `sys.argv[1:]`.
Exact output: every space, newline, punctuation matters; avoid extra prompts unless shown in spec.
Partial credit: readable names, helper funcs, comments for nontrivial logic, edge cases handled.

BUILT-INS

Built-in	Use/caution
`len(x)`	size; works if `__len__`
`range(n)`	0..n-1, not list in Py3
`enumerate(xs,1)`	line numbers/positions
`zip(a,b)`	stops at shortest
`sorted(xs)`	new list; `sort()` mutates
`sum(xs)`	numeric iterable; empty=0

String	Meaning
`strip()`	remove surrounding whitespace
`split(sep)`	list of parts
`join(parts)`	separator joins strings
`lower()`	case-normalise
`isdigit()`	digits only, not negative/decimal
`replace(a,b)`	new string

List	Meaning
`append(x)`	add one item
`extend(xs)`	add many items
`pop(i)`	remove+return
`remove(x)`	first equal value
`index(x)`	position or error
`copy()`	shallow copy

ERROR FIX TEMPLATES

Syntax: missing colon after `if/for/while/def/class`; missing closing),], quote; bad indent.
NameError: variable/function not defined in current scope; spelling/case; defined after use; local not returned.
TypeError: wrong operator for type, wrong arg count, concat str+int, calling non-callable.
ValueError: type exists but value invalid, e.g. `int("apple")`.
IndexError: list/string index out of range; check bounds before indexing.
KeyError: dict key missing; use `if key in d` or `d.get(key,default)`.

When explaining: line X does Y; but spec requires Z; fix by ...; this works because ...

TRACE QUICKIES

Trace item	Write down
function call	args, local vars, return
loop	iter no, condition, changed vars
list mutation	object contents after operation
recursion	call depth + subproblem
file	line number + parsed tokens

Mutation trace: `a=b`; `b.append(3)` changes both views; `b=[3]` only rebinds b.
Branch trace: in if/elif chain only one branch runs; independent ifs can all run.
Loop trace: condition checked before each while iteration; for loop assigns next value automatically.

For flowchart questions include start/end, decision diamonds, loopback, and output node.

EDGE CASE BANK

Numbers: zero, negative, boundary threshold, huge value, float rounding, divide by zero.
Strings: empty, one char, spaces, punctuation, different case, repeated letters, delimiter missing/repeated.
Lists: empty, one item, all same, duplicates, already sorted, reverse sorted, aliasing.
Dicts: missing key, repeated update, insertion order irrelevant unless output sorted.
Files: missing file, empty file, blank line, invalid token, extra whitespace, last line no newline.
Classes: invalid constructor args, method before attr exists, shared class var, printing object.
Recursion: empty input, target direct, target absent, deepest path, no cycles vs cycles.

FILE PARSING BLUEPRINT

read: `try: f=open(path)` or `with open(path) as f`; catch only expected errors.
normalise: `line=line.strip()`; if blank either skip or error per spec.
tokenise: `parts=line.split(",")` or whitespace split; validate `len(parts)`.
convert: wrap casts in try; report line number and relevant field.
accumulate: count/sum/list/dict; preserve order if output needs it.
finish: handle no valid data before average; sort output only if spec asks.

Never let bad file input crash if prompt says robust; never silently ignore if prompt says raise.

OOP BLUEPRINT

class: define attributes in `__init__`; keep invariant after every method.

`super().__init__(...)` in subclass to initialise inherited fields.

Method types: instance method uses self; class method uses cls; static method has no self/cls.
Equality: check type/attributes; `return isinstance(other, C) and self.x==other.x`.
Display: `__str__` should return string, not print.
Composition: object contains list/dict of other objects; implement add/remove/search helpers.

RECURSION BLUEPRINT

Recursive answer must call itself; otherwise zero if prompt says recursive.

numeric: if n==0 return base; return combine(n, f(n-1)).
string/list: if len<=1 return base; compare head/tail; recurse on middle/rest.
network: if direct match return 0; for each child get d; if d!=-1 return d+1; return -1.
Cycles: if possible, pass visited set; if prompt says no cycles, state assumption.
Stack model: each call has its own locals; return values flow back upward.

PYTHON GOTCHAS

`list.sort()` returns None; use sorted(xs) if you need a new sorted list.
`dict.keys()` is a view; wrap `list(...)` if exact list needed.
`is` tests identity; use == for value except `is None`.
Default mutable arg: avoid `def f(xs=[])`; use None then create list.
Shadowing: don't name variables `list`, `str`, `sum`, `file`.
Integer division: / float result, // floor division, % remainder.
Open mode: `'w'` overwrites; `'a'` appends; `'r'` needs existing file.

ANSWER TEMPLATES

Explain output: first evaluate conditions/calls; then list printed lines; mention no output if only return.
Fix code: quote faulty line, correction, and reason; preserve intended structure where possible.
Write function: signature, guard cases, main loop/recursion, return; no global I/O unless required.
Design class: attrs, constructor, methods, special methods, demo objects, expected output.
Testing answer: tests should name input, expected result, and why it targets a branch/edge.

If unsure, state assumption explicitly and produce deterministic code under that assumption.

SORT/COUNT PATTERNS

Top-k words: normalise text; count dict; sort by `(-count, word)`; slice first k.

`items = sorted(counts.items(), key=lambda p: (-p[1], p[0]))`

Anagram: if library allowed, compare `sorted(a)==sorted(b)`; if not, compare char counts.
Second largest distinct: ignore values equal to current largest; maintain largest and second.
Do not mutate input unless asked: use `for x in numbers` not `numbers.remove(max)` when edge cases matter.
Stable order with counts: keep `order=[]`; when first seeing key append to order; later update count only.
Sorting output: if ties specified alphabetically, use tuple key; if ties unspecified, state chosen order.

MATRIX TASKS

Read grid: validate rows all same length unless ragged allowed; convert cell type.

`for r in range(rows): for c in range(cols): value = grid[r][c]`

Neighbour check: for dr,dc in dirs; nr=r+dr; nc=c+dc; require `0<=nr<rows` and `0<=nc<cols`.
Copy 2D: `new=[row.copy() for row in grid]`; shallow copy of outer list not enough for row mutation.
Coordinate output: be clear whether row/col starts at 0 or 1; follow question wording.

COMMAND-LINE

`import sys`; program name is `sys.argv[0]`; first user arg is `sys.argv[1]`.

Batch args: `for raw in sys.argv[1:]: try: x=float(raw) ... except ValueError: pass`.
Missing args: check length before indexing; prompt may require exit code, message, or silent termination.
Division parser: `parts=s.split("/")`; valid iff len(parts)==2 and both cast to numbers and denominator nonzero.
Output: one result per valid arg unless spec says collect then print list.

FORMAT STRINGS

`f"{name} got {mark:.1f}"` rounds/display 1 decimal; value unchanged.

Common specs: currency `:.2f`; percent multiply by 100 then format; align rarely needed.
String concat: convert non-str with `str(x)` or use f-string.
repr vs str: lists print with brackets/quotes; custom class needs `__str__`.

LIBRARY LIMITS

If allowed: `collections.Counter`, `math.sqrt`, `random.choice`, `sys.argv`.
If restricted: no modules except stated; no `in` except for loops; no slicing; no `find/count/index`.

Read restriction block twice; a working shortcut can still receive zero.

Manual count without Counter: loop dict update; manual max over dict items.
Manual substring no slicing: nested loop and char compare; break on mismatch.

MARKING LANGUAGE

Correctness: handles all specifie[illegible]urns/prints exact values.
Readability: names, decompositi[illegible]logic, clear comments only where useful.
Robustness: invalid input/file/em[illegible] to spec.
Traceability: explanation matche[illegible] final answer.
Completeness: all subparts answ[illegible]ain algorithm for partial marks.

MINI DRILLS

`[i for i in range(5) if i%2=`[illegible]

`"Python Programming".split("o")` → ['Pyth','n Pr','gramming'].

`data={'a':1}; list(data.keys())` → ['a'].

`try/finally`: finally runs whether exception happened or not.

`global`: needed to assign to global name inside function; avoid when possible.

`__len__`: enables len(obj); `__eq__`: enables obj1 == obj2.

FAST PSEUDOCODE

Tax band: check highest? easiest is ascending thresholds; return as soon as matching band found.
Guessing game: reveal string by loop over indices; if guessed char matches, copy char else old mask char.
Word puzzle: read candidate words, normalise, length filter, compare sorted letters/counts, print matches.
Student school: class has `students=[]`; `add_student`; `__len__` returns length.
Product hierarchy: base stores common attrs; subclass calls `super`; overridden display adds extra attrs.
Text report: punctuation removal, lower, split, count, sort by frequency desc then alpha.
All these patterns need exact function/class names if the prompt specifies names.

MORE MINI CASES

`bool([])=False`, `bool([0])=True`, `bool("False")=True`.

`range(2,8,2)` yields 2,4,6; `range(5,0,-2)` yields 5,3,1.

`"abc"[1:]` is "bc"; `"abc"[-1]` is "c"; index 3 errors.

`{ }.get("x",0)` returns 0; `{ }["x"]` raises KeyError.

`open(path,"w")` truncates file immediately; delay until output is valid.

`except Exception` catches too broadly; catch specific exceptions when possible.

EXAM SELF-CHECK

Before final answer: Did I answer every bullet? Did I use required filename/function/class?
For code: Are variables initialised before use? Does every branch return/print as required?
For loops: Does it terminate? Does it include/exclude endpoints correctly?
For files: Are invalid lines specified as skip, error, or exception? Is empty data handled?
For OOP: Are attributes on `self`? Are special methods named exactly with double underscores?
For recursion: Is there a base case, smaller recursive call, and correct combine?

If code cannot be fully written, give clear pseudocode + tests for partial credit.

LAST-MINUTE RULES

Read the full spec before coding; later bullets often override earlier assumptions.
Keep helper functions small: one parse helper, one validation helper, one output helper.
Prefer returning data from helpers; print only in main/output layer unless prompt asks.
Check indentation carefully after every colon; one wrong indent can change the algorithm.
When tracing, do not simplify by intuition; execute line-by-line with current values.
When using lists/dicts in classes, initialise them inside `__init__`, not as shared class attrs.
For exact prompts, preserve names: `process_data`, `calculate`, `analyze_text_file`.
For answer explanations, tie every claim to a Python rule or a line of code.

Best exam answer = correct code + why it works + edge cases handled.

TINY RUBRIC

HD: solves general case, clean design, precise edge handling, explanation clear.
D: mostly correct, minor edge/format issue, readable.
CR: core idea present, some missing cases or small syntax errors.
P: partial algorithm, limited tests, many details missing.
Always leave marker something checkable: function signature, loop invariant, sample trace.